BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI
BỘ MÔN TRÍ TUỆ NHÂN TẠO

-------- --------

BÁO CÁO BÀI TẬP LỚN CUỐI KỲ

HỌC PHẦN: NHẬN DẠNG MẪU

NGHIÊN CỨU VÀ SO SÁNH CÁC PHƯƠNG PHÁP NHẬN DẠNG CHỮ SỐ VIẾT TAY TRÊN BỘ DỮ LIỆU MNIST

Giảng viên hướng dẫn	**ThS. Nguyễn Thị Hồng Tú**
Họ và tên sinh viên	**Đoàn Anh Vũ** **Nguyễn Hồng Nguyên**
Mã sinh viên	**2351267280** **2351267273**
Lớp	**S26-65TTNT**

TP. Hồ Chí Minh, 11/2026

NHẬN XÉT CỦA GIẢNG VIÊN

....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
.....

....
....
....
.....

....
....
....
....
....
....
.....

Nngày...tháng……năm 2026

Người nhận xét

(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Trong quá trình thực hiện bài tập lớn học phần Nhận dạng mẫu, nhóm chúng em đã vận dụng những kiến thức về xử lý và biểu diễn dữ liệu, học máy có giám sát và đánh giá mô hình để nghiên cứu bài toán nhận dạng chữ số viết tay. Nhóm xin bày tỏ lòng biết ơn chân thành tới giảng viên phụ trách học phần cùng quý thầy, cô Bộ môn Trí tuệ Nhân tạo đã trang bị nền tảng lý thuyết, phương pháp tiếp cận và định hướng học thuật cần thiết cho quá trình học tập và thực hiện đề tài.

Nhóm đồng thời trân trọng ghi nhận những đóng góp của các tác giả, nhà nghiên cứu và cộng đồng phát triển các phương pháp, bộ dữ liệu và thư viện được sử dụng trong đề tài; những nguồn tư liệu liên quan đều được dẫn trong phần Tài liệu tham khảo. Mặc dù đã nỗ lực kiểm tra, đối chiếu và phân tích kết quả thực nghiệm, báo cáo khó tránh khỏi những hạn chế. Nhóm kính mong nhận được ý kiến nhận xét và góp ý của giảng viên để tiếp tục hoàn thiện kiến thức chuyên môn và kỹ năng nghiên cứu.

Mục Lục

LỜI CẢM ƠN....3
TÓM TẮT....6
CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI....7
1.1. Bối cảnh và tính cấp thiết....7
1.2. Phát biểu bài toán....7
1.3. Mục tiêu và câu hỏi nghiên cứu....7
1.4. Phạm vi và đóng góp....8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÁC NGHIÊN CỨU LIÊN QUAN....9
2.1. Các công trình nền tảng và định hướng lựa chọn thuật toán....9
2.2. Biểu diễn dữ liệu và chuẩn hóa....9
2.3. Nắn nghiêng bằng mô-men ảnh và PCA....9
2.4. Naive Bayes....10
2.5. k-Nearest Neighbors....11
2.6. Support Vector Machine....11
2.7. Random Forest....12
2.8. Convolutional Neural Network....12
2.9. Thước đo đánh giá....12
CHƯƠNG 3. DỮ LIỆU, EDA VÀ TIỀN XỬ LÝ....13
3.1. Nguồn dữ liệu và kiểm tra chất lượng....13
3.2. Phân bố lớp và mức cân bằng....14
3.3. Phân bố pixel và đặc trưng hình dạng....15
3.4. Khảo sát PCA và t-SNE....16
3.5. Chia tập và kiểm soát rò rỉ dữ liệu....17
CHƯƠNG 4. THIẾT KẾ VÀ HUẤN LUYỆN CÁC MÔ HÌNH....18
4.1. Giao thức thực nghiệm chung....18
4.2. Cấu hình năm mô hình....18
4.3. Cấu hình huấn luyện CNN....18
4.4. Lưu kết quả và khả năng kiểm chứng....19
CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ THẢO LUẬN....20
5.1. Kết quả chung trên validation và test....20
5.2. Nhận xét theo từng phương pháp....20
5.3. Phân tích ma trận nhầm lẫn....21
5.4. Đường cong học và lựa chọn checkpoint CNN....24

5.5. Trả lời câu hỏi nghiên cứu.....25
CHƯƠNG 6. TRIỂN KHAI MINH HỌA, KHẢ NĂNG TÁI LẬP VÀ HẠN CHẾ.....26
6.1. Trạng thái ứng dụng và phạm vi được xác nhận.....26
6.2. Kiểm thử và đối chiếu kết quả.....26
6.3. Giới hạn của thực nghiệm.....26
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....27
TÀI LIỆU THAM KHẢO.....28

TÓM TẮT

Đề tài nghiên cứu và so sánh các phương pháp phân loại chữ số viết tay từ 0 đến 9 trên bộ dữ liệu MNIST. Dữ liệu gồm 70.000 ảnh xám kích thước 28 × 28 pixel; trong đó, 60.000 ảnh ban đầu được phân chia có phân tầng thành 50.000 ảnh huấn luyện và 10.000 ảnh kiểm định, còn 10.000 ảnh kiểm thử chính thức được giữ độc lập. Quy trình nghiên cứu bao gồm khám phá dữ liệu, chuẩn hóa và biểu diễn đặc trưng, huấn luyện và đánh giá năm mô hình: Bernoulli Naive Bayes, k-NN, SVM sử dụng nhân RBF, Random Forest và mạng nơ-ron tích chập (CNN). Đối với k-NN và SVM, ảnh được hiệu chỉnh độ nghiêng và giảm chiều bằng PCA với ngưỡng giữ lại 95% phương sai. Trên tập kiểm thử 10.000 ảnh, độ chính xác của năm mô hình theo thứ tự lần lượt đạt 84,46%; 98,06%; 98,79%; 96,99% và 99,44%. CNN đạt kết quả cao nhất với 56 ảnh nhận dạng sai; trọng số tốt nhất được lựa chọn theo sai số kiểm định tại epoch 20, trước khi quá trình huấn luyện dừng ở epoch 23. Báo cáo đồng thời phân tích các chỉ số Precision, Recall, F1-score, ma trận nhầm lẫn và diễn biến huấn luyện; từ đó thảo luận ưu điểm, hạn chế của các phương pháp và điều kiện triển khai thực tế. Kết quả phản ánh hiệu năng trong phạm vi phân loại chữ số đơn lẻ trên MNIST và chưa thể suy rộng sang mọi dạng ảnh viết tay ngoài bộ dữ liệu này.

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1. Bối cảnh và tính cấp thiết

Nhận dạng chữ số viết tay là một bài toán tiêu biểu của nhận dạng mẫu: từ một tín hiệu quan sát được là ảnh, hệ thống cần gán nhãn cho mẫu dựa trên các dấu hiệu học được trong dữ liệu. Bài toán đã được nghiên cứu từ lâu trong số hóa chứng từ, xử lý biểu mẫu và kiểm tra văn bản. Công trình của LeCun và cộng sự [1] góp phần hình thành cách tiếp cận học đặc trưng từ ảnh bằng mạng nơ-ron tích chập; trong khi đó, các thuật toán phân loại nền tảng như k-NN [2], SVM [3] và Random Forest [4] vẫn là những đối chứng có ý nghĩa để đánh giá sự phù hợp giữa biểu diễn đặc trưng và bộ phân lớp.

MNIST là bộ dữ liệu chuẩn hóa gồm ảnh xám một chữ số, được dùng rộng rãi để thực nghiệm phân lớp và so sánh thuật toán [1], [5]. Tuy nhiên, độ chính xác cao trên MNIST không đồng nghĩa hệ thống có thể nhận dạng nhiều chữ số viết nối, văn bản tự do hay ảnh thu từ camera trong điều kiện khác biệt. Vì vậy, đề tài giới hạn rõ phạm vi ở ảnh chữ số đơn lẻ, kích thước 28 × 28 và giao thức đánh giá cố định.

Điểm nhấn của báo cáo không phải tuyên bố một mô hình mới mà là tổ chức một quy trình thực nghiệm có kiểm soát: phân tích dữ liệu, giải thích các biến đổi, thực hiện nhiều phương pháp phân loại, ghi nhận tham số và đánh giá trên cùng tập test. Hướng này phù hợp yêu cầu học phần Nhận dạng mẫu: hiểu nguyên lý toán học, thực thi, so sánh và nhận xét trên bằng chứng định lượng.

1.2. Phát biểu bài toán

Cho ảnh đầu vào $x \in [0,1]^{28\times 28}$ và nhãn y thuộc mười lớp từ 0 đến 9. Hàm phân loại f_θ ánh xạ ảnh đến một trong mười lớp. Bộ dữ liệu có nhãn D gồm N cặp ảnh và nhãn tương ứng. Mục tiêu thực nghiệm là tìm tham số và cấu hình có sai số thấp trên các ảnh chưa dùng huấn luyện.

$$\hat{y} = f_\theta(x), \hat{y} \in \{0,1,\ldots,9\}.$$

Bài toán này là phân lớp đa lớp có giám sát. Đầu ra cho mỗi ảnh là nhãn chữ số dự đoán, không phải vị trí hộp bao, chuỗi ký tự hay bản phiên âm văn bản. Khi sử dụng thuật toán truyền thống, ảnh được trải phẳng thành véc-tơ 784 chiều hoặc qua bước nắn nghiêng và PCA. Đối với CNN, cấu trúc không gian hai chiều của ảnh được giữ lại để mạng học đặc trưng theo vùng lân cận.

1.3. Mục tiêu và câu hỏi nghiên cứu

Mục tiêu tổng quát là khảo sát ảnh hưởng của lựa chọn bộ phân lớp và biểu diễn dữ liệu đến chất lượng nhận dạng MNIST, từ đó chọn một mô hình có hiệu năng tốt trong phạm vi bộ thực nghiệm của nhóm. Các mục tiêu cụ thể gồm kiểm tra dữ liệu và mức cân bằng lớp; trình bày, áp dụng các bước tiền xử lý; huấn luyện năm mô hình; phân tích Accuracy, macro Precision, macro Recall, macro F1 và số ảnh nhận sai; đồng thời xem xét việc triển khai mô hình thành sản phẩm minh họa.

Ba câu hỏi được sử dụng để tổ chức lập luận: RQ1 – dữ liệu MNIST có đặc tính phân bố, cường độ và biến thiên hình dạng như thế nào, và PCA phản ánh gì về biểu diễn đặc trưng? RQ2

– với giao thức 50.000 train/10.000 validation/10.000 test, năm bộ phân lớp cho kết quả nào và khác biệt thể hiện ở những loại lỗi gì? RQ3 – lựa chọn CNN so với các thuật toán truyền thống có cơ sở nào trong kết quả thực nghiệm và những giới hạn nào chưa được kiểm chứng?

1.4. Phạm vi và đóng góp

Nghiên cứu được triển khai theo hai giai đoạn: phân tích khám phá dữ liệu MNIST và thực nghiệm huấn luyện, đánh giá các thuật toán phân lớp. Kết quả so sánh chính được xác định trên cùng một tập kiểm thử độc lập gồm 10.000 ảnh, với các bước tiền xử lý và cấu hình mô hình được mô tả tại những mục tương ứng. Các chỉ số được tổng hợp từ kết quả thực nghiệm đã ghi nhận và đối chiếu với quy trình tính toán. Phạm vi báo cáo không bao gồm việc xác nhận tính tái lập qua nhiều lần huấn luyện độc lập hoặc kiểm chứng mọi cấu hình trong các phiên bản triển khai trước.

Đóng góp ở cấp bài tập lớn gồm: (i) mô tả minh bạch pipeline và ranh giới train–validation –test; (ii) đối chiếu năm thuật toán với biểu diễn dữ liệu phù hợp; (iii) phân tích kết quả và lỗi được minh họa bằng kết quả trực quan hóa thực nghiệm; (iv) chỉ ra yêu cầu đồng bộ tiền xử lý và mô hình khi xây dựng ứng dụng nhận dạng thực tế.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÁC NGHIÊN CỨU LIÊN QUAN

2.1. Các công trình nền tảng và định hướng lựa chọn thuật toán

LeCun và cộng sự (1998) [1] trình bày phương pháp học dựa trên gradient cho nhận dạng tài liệu và giải thích tính hiệu quả của CNN đối với biến thiên hình dạng hai chiều. Đây là cơ sở học thuật quan trọng cho việc bổ sung một mô hình học sâu vào nhóm đối chứng. Cover và Hart (1967) [2] nghiên cứu quy tắc láng giềng gần nhất, đặt cơ sở cho cách phân loại dựa vào các mẫu đã biết mà không cần xây dựng một hàm quyết định tham số toàn cục. Cortes và Vapnik (1995) [3] xây dựng SVM lề mềm, thích hợp để khảo sát ranh giới phân lớp phi tuyến khi kết hợp với nhân RBF. Breiman (2001) [4] đề xuất Random Forest, tổng hợp quyết định của nhiều cây nhằm giảm độ phụ thuộc vào một cây đơn lẻ.

Báo cáo sử dụng PCA theo tinh thần tổng quan của Jolliffe và Cadima [6]: giữ lại các hướng có phương sai lớn, nhưng không đồng nhất phương sai với mức thông tin phục vụ phân lớp. Phép chiếu t-SNE được dùng **chỉ để trực quan hóa** cấu trúc lân cận của dữ liệu có số chiều lớn, theo phương pháp van der Maaten và Hinton [7]; ảnh hai chiều t-SNE không phải là bộ đặc trưng đưa trực tiếp vào năm mô hình được đánh giá. Phân tích Precision, Recall và F1 được đặt trong bối cảnh các thước đo phân loại đa lớp như Sokolova và Lapalme đã hệ thống hóa [8].

Các công trình liên quan có kích thước train, kiến trúc, tăng cường dữ liệu và quy tắc chọn checkpoint khác nhau. Accuracy giữa các công trình vì thế chưa đủ cơ sở để xếp hạng trực tiếp. Phần này trình bày nền tảng phương pháp; phần thực nghiệm so sánh năm mô hình trong cùng giao thức trên 10.000 ảnh test.

2.2. Biểu diễn dữ liệu và chuẩn hóa

Một ảnh MNIST ban đầu được lưu dưới dạng ma trận cường độ nguyên không dấu $I \in \{0,\dots,255\}^{28\times 28}$. Bước chuẩn hóa sử dụng phép chia cố định cho 255, không phải chuẩn hóa z-score theo trung bình/độ lệch chuẩn. Phép biến đổi tạo miền giá trị nhất quán và giữ nguyên tỷ lệ tương đối của cường độ:

$$I_{norm}(u,v)=\frac{I(u,v)}{255}, 0\le I_{norm}(u,v)\le 1.$$

Với các bộ phân lớp truyền thống dùng điểm ảnh, ảnh có thể trải phẳng thành $x \in \mathbb{R}^{784}$. Việc trải phẳng là một phép sắp xếp lại dữ liệu, không tạo thêm thông tin; nhược điểm là thông tin lân cận không còn được mã hóa trực tiếp trong cấu trúc đầu vào. CNN nhận tensor một kênh kích thước $1\times 28\times 28$ để duy trì cấu trúc không gian.

2.3. Nắn nghiêng bằng mô-men ảnh và PCA

Deskew là phép hiệu chỉnh độ nghiêng của nét chữ dựa trên các mô-men của ảnh. Trong quá trình tiền xử lý, hàm cv2.moments được gọi trên từng ảnh; khi $|\mu_{02}|<0,01$, hệ số nghiêng được đặt bằng 0 để tránh chia cho mẫu số rất nhỏ. Trường hợp còn lại tính:

$$s=\frac{\mu_{11}}{\mu_{02}}, A=\begin{bmatrix}1 & s & -14\,s\\ 0 & 1 & 0\end{bmatrix}.$$

Ma trận A được đưa vào cv2.warpAffine với cờ WARP_INVERSE_MAP và INTER_LINEAR. Điều này quan trọng khi tái lập: quy ước ánh xạ của OpenCV phải trùng với cách tạo ma trận biến đổi; viết lại bằng một phép shear khác dấu có thể cho kết quả khác. Deskew chỉ được áp dụng cho nhánh k-NN và SVM trong cấu hình thực nghiệm, **không** áp dụng cho Naive Bayes, Random Forest và CNN.



Hình 2.1. Minh họa ảnh chữ số trước và sau hiệu chỉnh độ nghiêng bằng phép biến đổi affine.

Sau deskew, dữ liệu train được đưa qua PCA với ngưỡng giữ 95% phương sai. Với ma trận train đã trừ trung bình $X_c \in \mathbb{R}^{N \times D}$, ma trận hiệp phương sai mẫu là:

$$\Sigma = \frac{1}{N-1} X_c^T X_c, \Sigma v_j = \lambda_j v_j.$$

Nếu $\lambda_1 \geq \lambda_2 \geq \cdots$, số chiều giữ lại được chọn là số nguyên nhỏ nhất d thỏa:

$$\frac{\sum_{j=1}^{d} \lambda_j}{\sum_{j=1}^{D} \lambda_j} \geq 0.95.$$

Trong **EDA**, PCA fit trên toàn bộ 60.000 ảnh train gốc cho kết quả 154 chiều. Trong **huấn luyện**, PCA được fit trên 50.000 ảnh train sau deskew và còn 113 chiều. Hai con số không mâu thuẫn vì khác đầu vào và khác tập fit. Trên validation và test chỉ gọi transform, không gọi fit lại. Đây là quy tắc quan trọng để tránh rò rỉ thông tin từ tập đánh giá [9].

2.4. Naive Bayes

Naive Bayes giả định các đặc trưng độc lập có điều kiện theo nhãn. Cấu hình thực nghiệm sử dụng BernoulliNB(alpha=1.0, binarize=0.2) trên 784 pixel đã được chuẩn hóa; tức các pixel liên tục được nhị phân hóa theo ngưỡng và mô hình hóa sự hiện diện của nét. Quy tắc phân lớp có dạng:

$$\hat{y} = argmax_{k \in \{0,\ldots,9\}} \left[\log P(Y=k) + \sum_{j=1}^{784} \log P(X_j = x_j \mid Y=k) \right].$$

Ưu điểm là huấn luyện đơn giản, chi phí thấp và tạo được đối chứng nền tảng. Hạn chế là các pixel kề nhau trong ảnh chữ số có tương quan mạnh, vi phạm xấp xỉ độc lập có điều kiện. Mô hình vì thế có thể bỏ qua cấu trúc nét và hình học mà các phương pháp khác tận dụng tốt hơn.

2.5. k-Nearest Neighbors

k-NN gán nhãn theo k mẫu train gần nhất trong không gian đặc trưng. Cấu hình khảo sát sử dụng $k = 3$, khoảng cách Euclid và trọng số theo khoảng cách. Với hai véc-tơ x và x_i trong không gian PCA, khoảng cách được tính bằng:

$$d(x, x_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}.$$

Sau khi tìm tập $N_k(x)$ gồm ba láng giềng, nhãn được chọn bằng cách cộng trọng số của những láng giềng cùng lớp:

$$\hat{y} = \mathrm{argmax}_c \sum_{i \in N_k(x)} 1(y_i = c) w_i, w_i \propto \frac{1}{d(x, x_i)}.$$

Cách tính trọng số khi khoảng cách đúng bằng 0 được xử lý bởi triển khai scikit-learn, vì vậy biểu thức tỉ lệ trên là mô tả nguyên lý, không phải bản sao chính xác mọi nhánh xử lý số học của thư viện. k-NN không huấn luyện một tập trọng số theo gradient nhưng vẫn cần lưu dữ liệu tham chiếu. Điều này có thể làm chi phí suy luận tăng khi số mẫu train lớn [2], [10].

2.6. Support Vector Machine

SVM tìm siêu phẳng phân biệt các lớp với lề lớn. Với dữ liệu không phân tách tuyến tính, bài toán lề mềm nhị phân có thể được mô tả:

$$\min_{w,b,\xi} \frac{1}{2} \| w \|_2^2 + C \sum_{i=1}^{N} \xi_i$$

$$\text{s.t. } y_i \left(w^T \phi(x_i) + b \right) \geq 1 - \xi_i, \xi_i \geq 0.$$

Thực nghiệm sử dụng SVC(C=5, kernel='rbf', gamma='scale') và đầu vào deskew + PCA 113 chiều. Nhân RBF có biểu thức:

$$K(x, z) = \exp\left(-\gamma \| x - z \|_2^2 \right).$$

Với lựa chọn gamma='scale', scikit-learn xác định γ dựa trên số đặc trưng và phương sai dữ liệu nhận vào, thay vì sử dụng một hằng số do nhóm đặt thủ công. Đối với phân lớp nhiều nhãn, triển khai SVC giải quyết các bài toán nhị phân theo chiến lược one-vs-one. SVM là phương pháp đối chứng quan trọng bởi nó kết hợp biểu diễn hình học đã được giảm chiều với ranh giới quyết định phi tuyến [3], [10].

2.7. Random Forest

Random Forest kết hợp nhiều cây quyết định, mỗi cây học từ mẫu dữ liệu và lựa chọn tập đặc trưng ngẫu nhiên ở các nút phân chia [4]. Nhóm sử dụng 150 cây, max_features='sqrt', max_depth=None và random_state=42. Độ hỗn tạp Gini tại nút có phân bố lớp p_c là:

$$G = 1 - \sum_{c=0}^{9} p_c^2.$$

Random Forest chọn phép tách theo mức giảm Gini và tổng hợp dự đoán của 150 cây. Trong thực nghiệm, mô hình dùng 784 pixel chuẩn hóa, còn k-NN/SVM dùng Deskew + PCA. Chênh lệch kết quả vì thế phản ánh cả tiền xử lý và thuật toán. Random Forest học được ranh giới phi tuyến nhưng không biểu diễn trực tiếp cấu trúc lân cận hai chiều của ảnh.

2.8. Convolutional Neural Network

CNN dùng các kernel tích chập chia sẻ trọng số để trích xuất mẫu cục bộ trong ảnh [1], [11]. Mô hình được xây dựng có ba tầng tích chập: $1 \to 32$, $32 \to 64$ và $64 \to 128$ kênh, kết hợp BatchNorm, ReLU và MaxPool. Đầu ra được trải phẳng đến 512 phần tử, qua Dropout 0,4, lớp đầy đủ $512 \to 128$, ReLU, Dropout 0,2 và lớp cuối $128 \to 10$. CNN sử dụng cross-entropy trên logits và cập nhật trọng số bằng Adam. Với vector logits $z_i \in \mathbb{R}^{10}$, hàm mất mát cho tập mini-batch kích thước B là:

$$\mathcal{L}_{CE} = -\frac{1}{B}\sum_{i=1}^{B} \log\left(\frac{e^{z_{i,y_i}}}{\sum_{c=0}^{9} e^{z_{i,c}}}\right).$$

BatchNorm và Dropout hỗ trợ quá trình tối ưu, nhưng không bảo đảm loại bỏ hoàn toàn overfitting. Việc lựa chọn checkpoint tốt nhất dựa trên **validation loss** thay vì test accuracy giúp duy trì một ranh giới hợp lý giữa điều chỉnh mô hình và đánh giá cuối cùng. Triển khai sử dụng thư viện PyTorch [12].

2.9. Thước đo đánh giá

Accuracy là tỉ lệ dự đoán đúng trên toàn bộ tập test; số lỗi tương ứng là $N_{error} = N_{test} - N_{correct}$. Với từng lớp c, ký hiệu TP_c, FP_c và FN_c lần lượt là đúng dương, dương giả và âm giả theo cách xem lớp c là dương tính:

$$Accuracy = \frac{\sum_{i=1}^{N} 1\left(\hat{y}_i = y_i\right)}{N}, Precision_c = \frac{TP_c}{TP_c + FP_c}, Recall_c = \frac{TP_c}{TP_c + FN_c}.$$

$$F1_c = \frac{2\,Precision_c\,Recall_c}{Precision_c + Recall_c}; F1_{macro} = \frac{1}{10}\sum_{c=0}^{9} F1_c.$$

Macro Precision và Recall lấy trung bình đều trên 10 lớp [8]. Quy tắc zero_division=0 xử lý mẫu số bằng 0. Ma trận nhầm lẫn đếm mẫu theo nhãn thật và nhãn dự đoán; tổng đường chéo là số mẫu đúng.

CHƯƠNG 3. DỮ LIỆU, EDA VÀ TIỀN XỬ LÝ

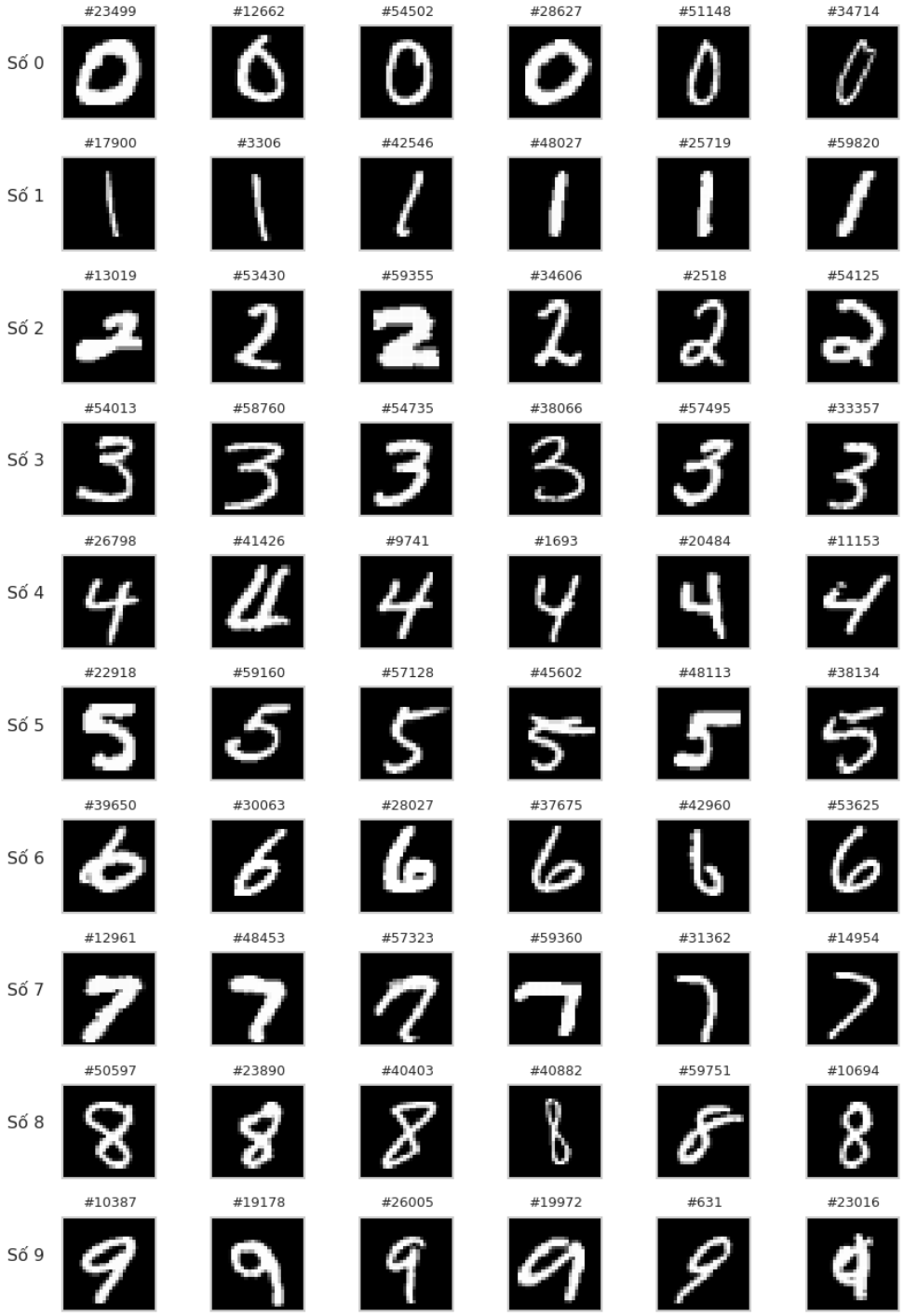
3.1. Nguồn dữ liệu và kiểm tra chất lượng

MNIST chứa 70.000 ảnh xám, gồm 60.000 ảnh train gốc và 10.000 ảnh test chính thức. Mỗi ảnh kích thước 28 × 28, nhãn nguyên 0–9. Dữ liệu được nạp theo định dạng phân phối của Keras/TensorFlow; nguồn dữ liệu và mô tả định dạng có thể kiểm tra tại tài liệu Keras [5]. Dữ liệu đầu vào có giá trị cường độ pixel 0–255 và kiểu uint8.

Bảng 3.1. Cấu trúc dữ liệu đầu vào trong giai đoạn khảo sát

Tập dữ liệu	Số ảnh	Kích thước	Kiểu pixel	Số lớp
Train gốc	60.000	28 × 28	uint8	10
Test chính thức	10.000	28 × 28	uint8	10

Kết quả kiểm tra ở giai đoạn phân tích khám phá dữ liệu cho thấy cả hai tập không có NaN trong ảnh hoặc nhãn, không có ảnh trống hoàn toàn, nhãn không vượt ngoài khoảng 0–9 và cường độ pixel ghi nhận từ 0 đến 255. Kết quả này chỉ xác nhận **những điều kiện đã kiểm tra**, không chứng minh toàn bộ nhãn đều chính xác hoặc dữ liệu không có mẫu viết gây tranh cãi.



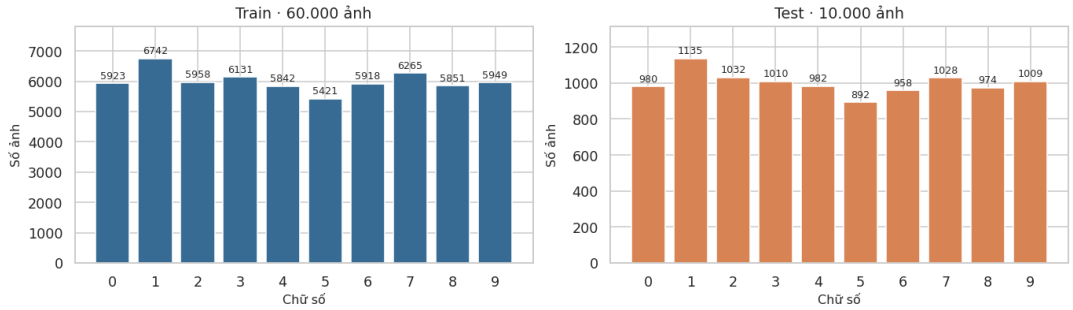
Hình 3.1. Ví dụ chữ số viết tay thuộc mười lớp.

3.2. Phân bố lớp và mức cân bằng

Kết quả đếm mẫu cho thấy lớp 1 có số lượng lớn nhất (6.742 ảnh train) và lớp 5 ít nhất (5.421 ảnh train). Chênh lệch này tạo phân bố gần cân bằng nhưng không hoàn toàn đồng đều. Tỷ số số mẫu nhiều nhất/ít nhất trên train là 1,2437. Shannon entropy của nhãn, với xác suất theo lớp p_c, được xác định bằng:

$$H(Y) = -\sum_{c=0}^{9} p_c \log_2 p_c, H_{max} = \log_2 10 \approx 3.3219\, bit.$$

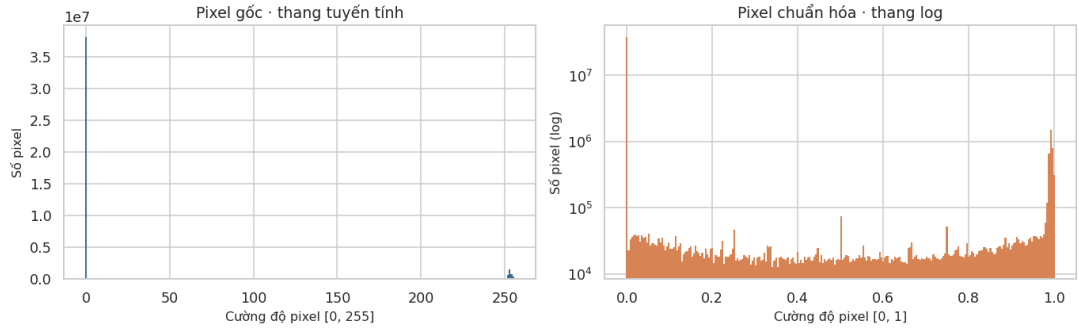
Entropy tập train đạt 3,3199 bit và tập test đạt 3,3194 bit theo kết quả phân tích. Những giá trị này gần cực đại của mười lớp nhưng không có nghĩa mọi lớp có đúng cùng số mẫu. Việc phân tầng train–validation giúp giữ gần đúng tỷ lệ nhãn, giảm rủi ro một lớp ít mẫu bị giảm quá mạnh ở tập huấn luyện.



Hình 3.2. Phân bố số lượng ảnh theo lớp ở train và test.

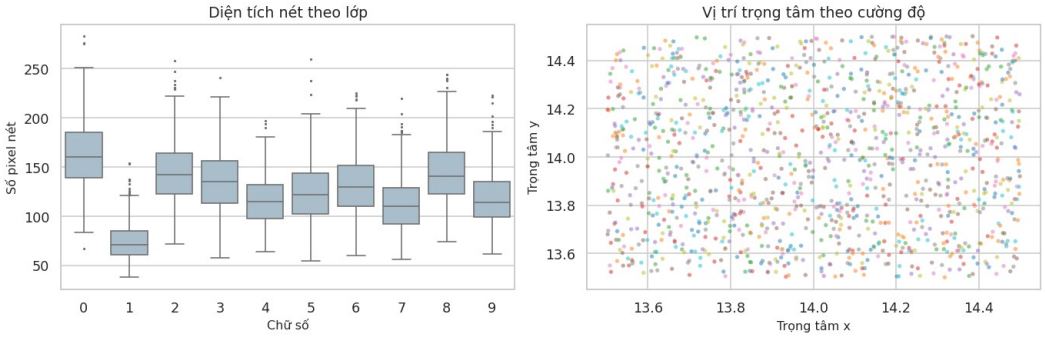
3.3. Phân bố pixel và đặc trưng hình dạng

Khoảng 80,8798% pixel trong train có giá trị 0, cho thấy phần lớn diện tích ảnh là nền đen; 0,6681% đạt giá trị 255. Giá trị trung bình pixel sau chuẩn hóa là 0,1307. Vì nền chiếm tỷ lệ cao, hai ảnh có thể giống nhau đáng kể ở vùng nền dù hình dạng nét khác nhau; điều này là một lý do cần kết hợp kiến thức về hình học hoặc học đặc trưng phù hợp, thay vì chỉ dựa vào từng điểm ảnh như biến độc lập.



Hình 3.3. Phân bố cường độ pixel trên tập train.

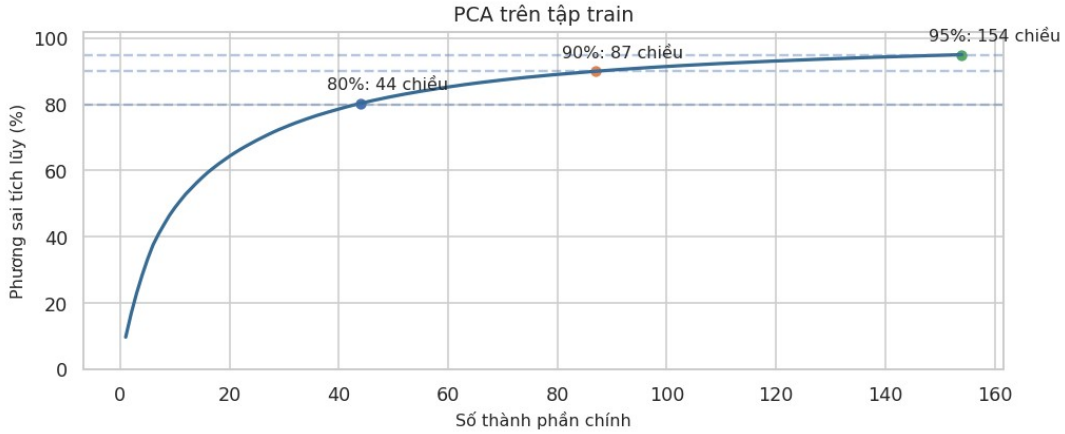
Nghiên cứu khảo sát thêm diện tích nét và trọng tâm cường độ trên mẫu ngẫu nhiên 5.000 ảnh train. Diện tích nét dùng ngưỡng pixel lớn hơn 0,2 trên ảnh chuẩn hóa; tọa độ trọng tâm được lấy trung bình có trọng số bởi cường độ. Các phép thống kê này dùng để **mô tả biến thiên**, không phải đặc trưng đầu vào bổ sung trong quá trình huấn luyện. Hộp phân bố cho thấy độ rộng nét và vị trí trọng tâm thay đổi giữa các mẫu cùng lớp và giữa các lớp.



Hình 3.4. Phân bố diện tích nét và trọng tâm cường độ.

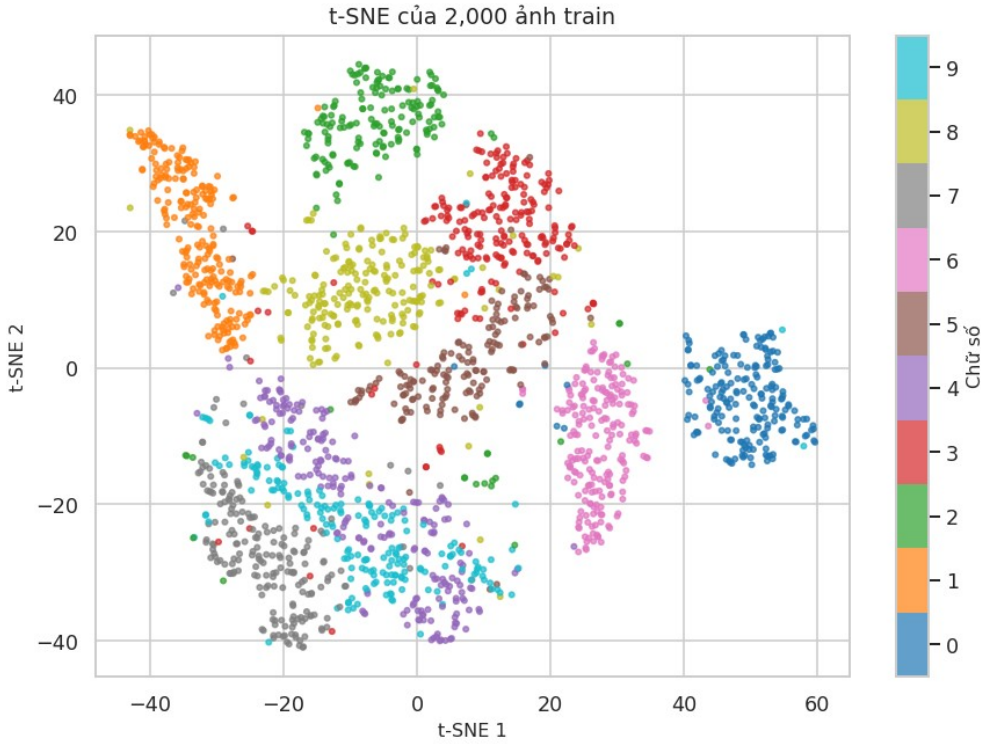
3.4. Khảo sát PCA và t-SNE

Trong EDA, PCA được fit trên tập train gốc sau phép chia 255, chưa deskew. Để đạt ngưỡng 95% phương sai, 784 chiều được rút xuống 154 chiều. Điều này cho thấy dữ liệu có nhiều hướng biến thiên tương quan nhưng không chứng minh 154 thành phần là tối ưu cho mọi mô hình. Hai thành phần đầu chỉ phục vụ minh họa; các đám mây điểm có chồng lấn khi nhìn ở hai chiều không đủ để kết luận về khả năng phân lớp ở không gian cao hơn.



Hình 3.5. Số thành phần PCA cần giữ theo ngưỡng phương sai.

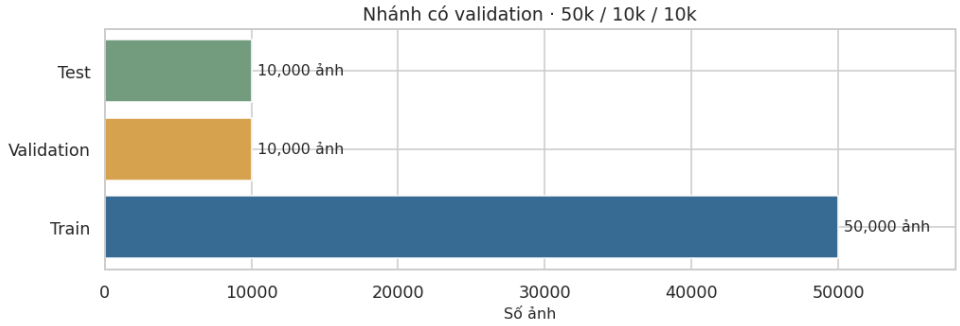
t-SNE được chạy trên 2.000 ảnh train; dữ liệu qua PCA còn 50 chiều, sau đó t-SNE giảm xuống hai chiều với perplexity 30 và seed 42. Biểu đồ thể hiện cấu trúc lân cận giữa các mẫu, còn khoảng cách toàn cục giữa các cụm chưa đại diện cho khoảng cách phân lớp định lượng [7]. Chất lượng bộ phân loại được đánh giá bằng các chỉ số trên tập test.



Hình 3.6. Trực quan t-SNE trên 2.000 ảnh train.

3.5. Chia tập và kiểm soát rò rỉ dữ liệu

Quá trình huấn luyện sử dụng train_test_split(..., test_size=10000, stratify=labels, random_state=42) để phân hoạch 60.000 ảnh phát triển thành 50.000 train và 10.000 validation. Tập test chính thức 10.000 ảnh được giữ tách biệt. PCA của nhánh deskew fit duy nhất bằng train 50.000 ảnh, sau đó chuyển đổi validation/test bằng cùng phép biến đổi. Dữ liệu test được sử dụng để ghi nhận chỉ số cuối cùng; quy trình hiện tại chưa thể hiện một quy trình tìm kiếm siêu tham số có hệ thống với nhiều cấu hình trước khi chốt báo cáo.



Hình 3.7. Phân chia dữ liệu trong giao thức thực nghiệm.

Điều quan trọng trong đọc kết quả là **không ghép** kích thước PCA từ EDA (154 chiều, train gốc 60.000) với nhánh k-NN/SVM trong giai đoạn huấn luyện (113 chiều, sau deskew và fit trên 50.000). Hai bước có cùng tên PCA nhưng có điều kiện khác nhau. Nói cách khác, EDA tạo bằng chứng mô tả cấu trúc dữ liệu; giai đoạn huấn luyện xác định pipeline dùng để lấy kết quả phân loại.

CHƯƠNG 4. THIẾT KẾ VÀ HUẤN LUYỆN CÁC MÔ HÌNH

4.1. Giao thức thực nghiệm chung

Tất cả năm mô hình dùng 50.000 ảnh train, 10.000 ảnh validation và 10.000 ảnh test chính thức. Seed chung là 42. CNN chạy trên CPU với 4 luồng; môi trường kiểm tra sử dụng Python 3.12, PyTorch 2.15.0+cpu, NumPy 2.5.4 và scikit-learn 1.8.0. Các phiên bản thư viện được ghi trong requirements-verified.txt. Notebook 02 thực hiện huấn luyện trực tiếp; script train_notebook_models.py chạy cùng các cell của notebook và lưu kết quả vào saved_models/.

Quy trình được tổ chức: nạp ảnh và nhãn → kiểm tra hình dạng → chia train/validation → chuẩn hóa → chuẩn bị nhánh biểu diễn theo mô hình → fit trên train → đo validation → ghi đầu ra test → lưu mô hình, dự đoán và chỉ số. Tuy dùng chung phân hoạch và tập test, năm cấu hình không dùng cùng một biểu diễn đầu vào, nên khác biệt kết quả phản ánh **tổ hợp tiền xử lý + thuật toán**. Muốn đo đóng góp riêng của deskew/PCA phải thực hiện phép đối chứng giữ nguyên thuật toán rồi thay một biến tại một thời điểm.

4.2. Cấu hình năm mô hình

Bảng 4.1. Tham số có thể đối chiếu trực tiếp

Phương pháp	Đầu vào	Cấu hình chính
Naive Bayes	784 pixel chuẩn hóa	BernoulliNB; alpha=1,0; binarize=0,2
k-NN	Deskew → PCA 113 chiều	3 láng giềng; weights=distance
SVM	Deskew → PCA 113 chiều	C=5; kernel=RBF; gamma=scale
Random Forest	784 pixel chuẩn hóa	150 cây; max_features=sqrt; không đặt max_depth
CNN	Tensor 1 × 28 × 28	3 khối tích chập; Adam; Cross-Entropy

Naive Bayes là đối chứng xác suất đơn giản. k-NN thể hiện phương pháp học dựa trên mẫu có sẵn. SVM đại diện cho bộ phân lớp lề lớn phi tuyến. Random Forest cung cấp một mô hình tập hợp của cây quyết định. CNN được đưa vào để khảo sát cách học trực tiếp các đặc trưng cục bộ trên ảnh. Lựa chọn năm phương pháp giúp bài làm vượt yêu cầu tối thiểu hai thuật toán và vẫn giữ ít nhất một thuật toán nền tảng trong học phần.

4.3. Cấu hình huấn luyện CNN

CNN được khởi tạo với seed 42; số tham số được in từ mã model sau khi khởi tạo. Mỗi epoch chạy qua train loader có xáo trộn mini-batch 64 mẫu, sau đó đo validation trên batch 256. Optimizer Adam có learning rate ban đầu 0,001 và weight decay 0,0001. Lịch giảm tốc độ học dùng ReduceLROnPlateau theo validation loss, hệ số giảm 0,5, patience 1 và ngưỡng dưới learning rate 10^{-5}.

Quy tắc dừng sớm theo dõi cải thiện của validation loss với patience=3 và min_delta=0,0001. Mã ghi giới hạn epoch khởi tạo 70 và khả năng gia hạn thêm 20 epoch khi chạm giới hạn, nhưng lần chạy đã lưu thực tế dừng ở epoch 23, nên giới hạn 70 **không** phải số epoch đã thực hiện. Checkpoint tốt nhất được chọn ở epoch 20 theo validation loss = 0,0218;

khi dừng tại epoch 23, trọng số được khôi phục từ epoch 20 trước khi đánh giá validation và test.

Bảng 4.2. Các thông số điều khiển CNN

Hạng mục	**Giá trị sử dụng**
Optimizer	Adam
Learning rate ban đầu	0,001
Weight decay	0,0001
Batch train / validation	64 / 256
Early stopping	patience 3; min_delta 0,0001
Giảm LR	factor 0,5; patience 1; min LR 0,00001
Epoch dừng	23
Epoch checkpoint tốt nhất	20
Validation loss tại checkpoint	0,0218

4.4. Lưu kết quả và khả năng kiểm chứng

Bộ kết quả trong saved_models/ gồm bốn mô hình ML dạng joblib, trọng số CNN.pt, bộ PCA, lịch sử cnn_history.csv, dự đoán predictions.npz, bảng results.csv và cấu hình selected_config.json. Các tệp mô tả cùng một lần thực nghiệm với phân hoạch 50.000 train, 10.000 validation và 10.000 test. Ứng dụng web nạp trực tiếp checkpoint và PCA từ thư mục này. Các chỉ số được tính lại từ dự đoán để đối chiếu với kết quả trong notebook.

Mã nguồn ghi seed, cấu hình mô hình, cách chia tập và phép biến đổi dữ liệu. Tệp requirements-verified.txt ghi phiên bản thư viện của môi trường kiểm tra. CNN được chạy lại theo mã notebook và tái hiện lịch sử 23 epoch, checkpoint epoch 20 cùng 56 ảnh sai trên test. Kết quả này xác nhận bộ tệp hiện tại; thay đổi thư viện, thiết bị hoặc seed có thể tạo một lần huấn luyện khác.

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ THẢO LUẬN

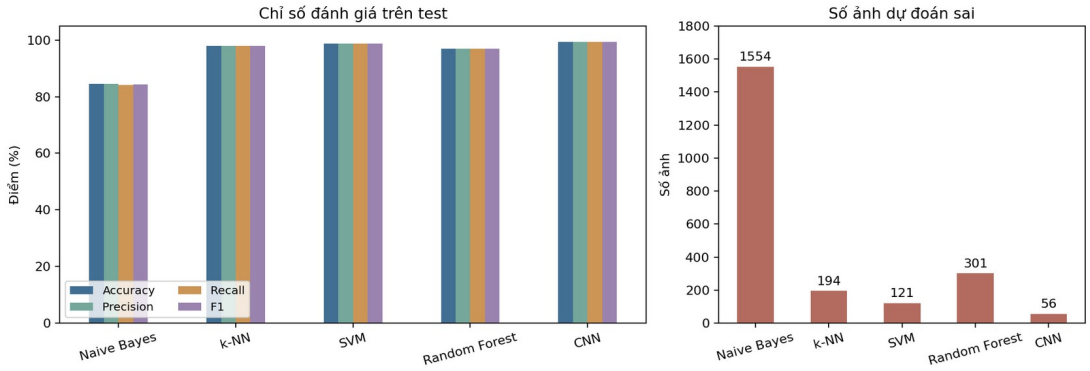
5.1. Kết quả chung trên validation và test

Bảng 5.1 tổng hợp từ kết quả đánh giá của năm mô hình trong quá trình huấn luyện. Mọi model đều được chấm trên 10.000 ảnh test chính thức. Precision, Recall và F1 là trung bình macro theo mười lớp. Kết quả validation đóng vai trò quan sát trong quá trình phát triển, không thay thế test accuracy.

Bảng 5.1. So sánh năm phương pháp

Mô hình	Val Acc (%)	Test Acc (%)	Precision macro (%)	Recall macro (%)	F1 macro (%)	Số ảnh sai
Naive Bayes	83,08	84,46	84,58	84,20	84,27	1.554
k-NN	97,93	98,06	98,06	98,04	98,05	194
SVM RBF	98,68	98,79	98,78	98,79	98,78	121
Random Forest	96,73	96,99	96,98	96,97	96,97	301
CNN	**99,41**	**99,44**	**99,43**	**99,43**	**99,43**	**56**

CNN dẫn đầu về cả Test Accuracy và số lượng ảnh được nhận đúng trong lần chạy lưu. Độ chính xác 99,44% tương ứng 9.944/10.000 ảnh đúng và 56 ảnh sai. SVM đứng thứ hai với 9.879 ảnh đúng. Khoảng cách giữa hai mô hình bằng 0,65 điểm phần trăm hoặc 65 ảnh nhận đúng khác nhau trên tập 10.000 mẫu, chưa phải kết luận về mức ý nghĩa thống kê hay độ ổn định qua nhiều seed. k-NN đạt 98,06%, cho thấy một phương pháp dựa trên khoảng cách, khi có deskew và PCA, vẫn đạt hiệu quả đáng kể trên dữ liệu chữ số được chuẩn hóa.



Hình 5.1. Kết quả năm mô hình trên test chung.

5.2. Nhận xét theo từng phương pháp

Naive Bayes. Accuracy 84,46% và 1.554 lỗi là kết quả thấp nhất trong nhóm. Điều này phù hợp với hạn chế mô hình hóa riêng rẽ từng pixel đã nhị phân hóa: cách viết cùng chữ số có thể thay đổi vị trí, độ dày và hướng nét, làm cho quan hệ phụ thuộc theo không gian trở nên quan trọng. Tuy nhiên, đây chỉ là lời giải thích mang tính cơ chế; để xác định nguyên nhân từng lỗi cần kiểm tra ảnh lỗi và xác suất dự đoán của mô hình.

k-NN. Sau deskew và PCA 113 chiều, thuật toán đạt 98,06%. Lợi thế là lựa chọn nhãn dựa trên các ví dụ trực tiếp từ train và không phải học một ranh giới phân biệt dạng tham số. Đổi lại,

suy luận cần tìm láng giềng trên tập tham chiếu, nên để so sánh tốc độ vận hành cần đo độ trễ trên cùng phần cứng và batch size, điều chưa được đo lường trong thực nghiệm hiện tại.

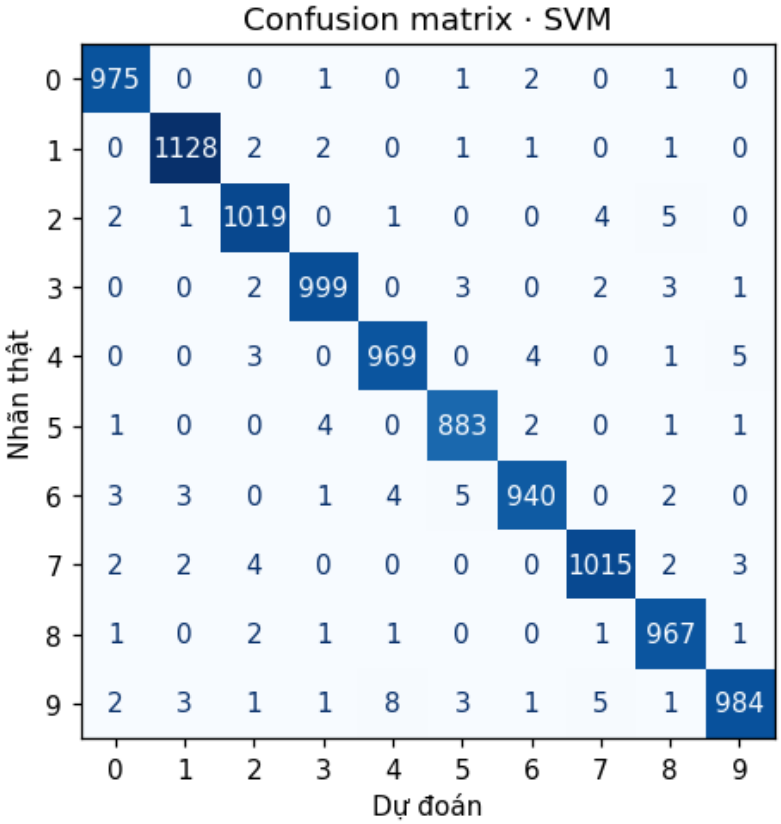
SVM. Cấu hình SVC nhân RBF với C=5 đạt 98,79%, tương ứng 121 ảnh sai. Đây là kết quả cao nhất trong nhóm mô hình cổ điển được khảo sát. Kết quả giới hạn ở MNIST với một phân hoạch và một cấu hình đại diện cho mỗi thuật toán. SVM và k-NN cùng sử dụng Deskew + PCA, nhưng có cơ chế quyết định khác nhau.

Random Forest. Với 150 cây, mô hình đạt 96,99%. Kết quả thấp hơn k-NN và SVM trong cấu hình này, nhưng Random Forest sử dụng ảnh thô đã chuẩn hóa chứ không dùng deskew + PCA. Nếu muốn so sánh tuyệt đối năng lực mô hình, cần thêm các nhánh Random Forest với tiền xử lý tương đương và điều chỉnh siêu tham số trên validation.

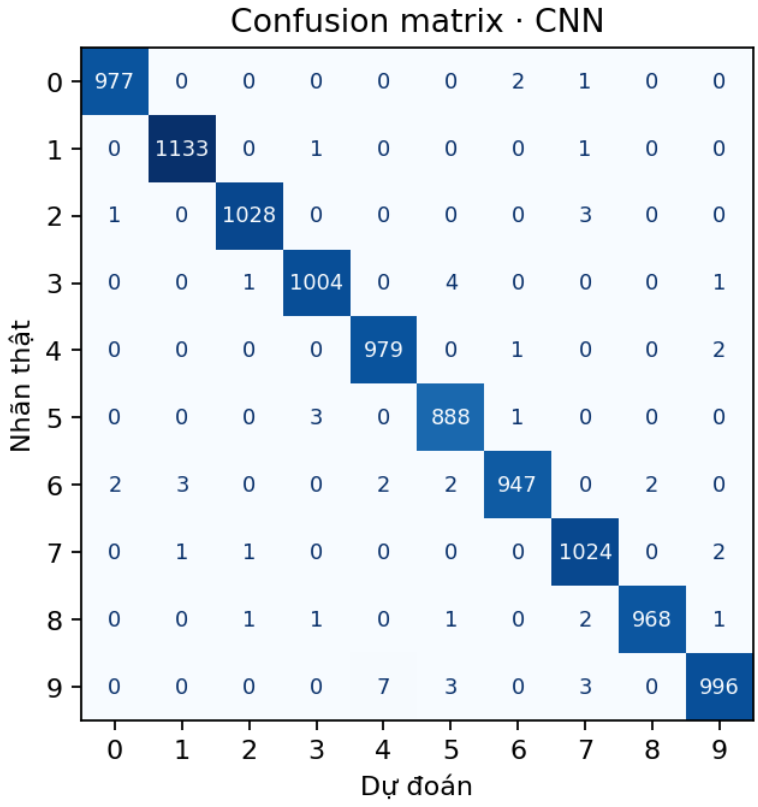
CNN. Việc học bộ lọc cục bộ giúp CNN đạt 99,44% trong lần chạy này. Đây cũng là mô hình duy nhất dùng kiến trúc 2D thay vì vector hóa để biểu diễn ảnh ngay từ đầu. Đường học và ma trận nhầm lẫn cần được xem cùng các chỉ số; đặc biệt, độ chính xác trên test vẫn không tự bảo đảm khả năng khái quát sang ảnh điện thoại, hình vẽ bằng chuột hay chữ số ngoài MNIST.

5.3. Phân tích ma trận nhầm lẫn

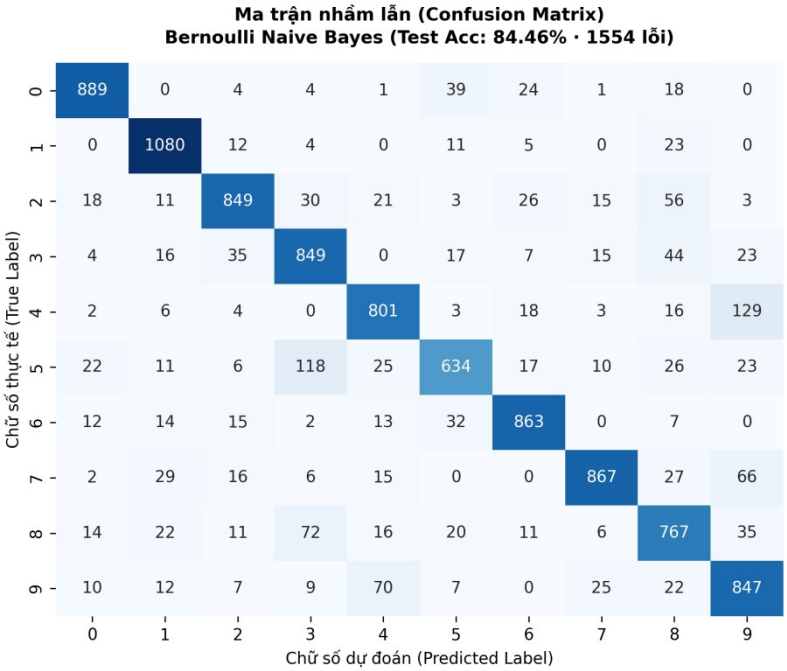
Ma trận nhầm lẫn cho thấy mô hình có thể đạt Accuracy cao nhưng vẫn nhầm ở các cặp số có hình dạng gần nhau. Ở CNN, đường chéo lần lượt có 977 ảnh lớp 0, 1.133 ảnh lớp 1, 1.028 ảnh lớp 2, 1.004 ảnh lớp 3, 979 ảnh lớp 4, 888 ảnh lớp 5, 947 ảnh lớp 6, 1.024 ảnh lớp 7, 968 ảnh lớp 8 và 996 ảnh lớp 9 nhận đúng. Tổng các phần tử đường chéo bằng 9.944, đúng với bảng kết quả.



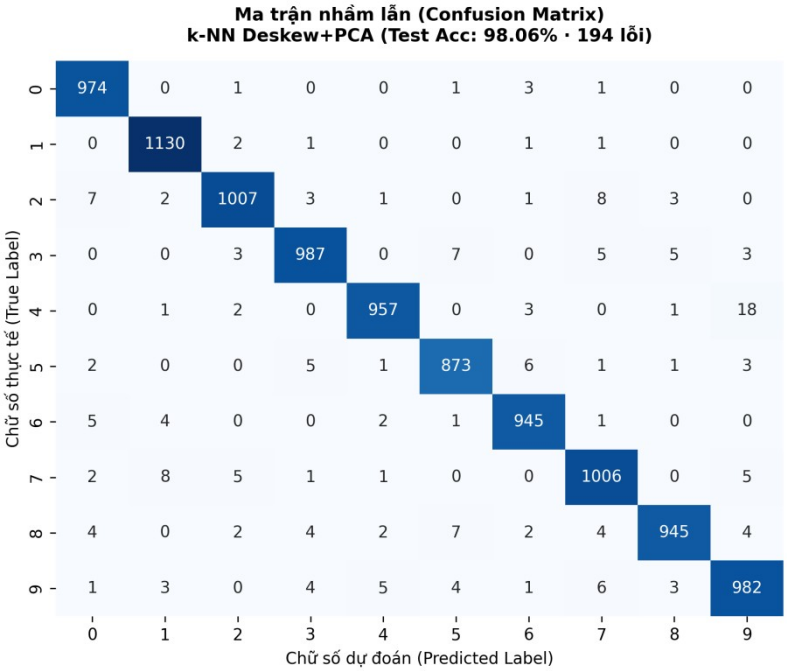
Hình 5.2. Ma trận nhầm lẫn của SVM .



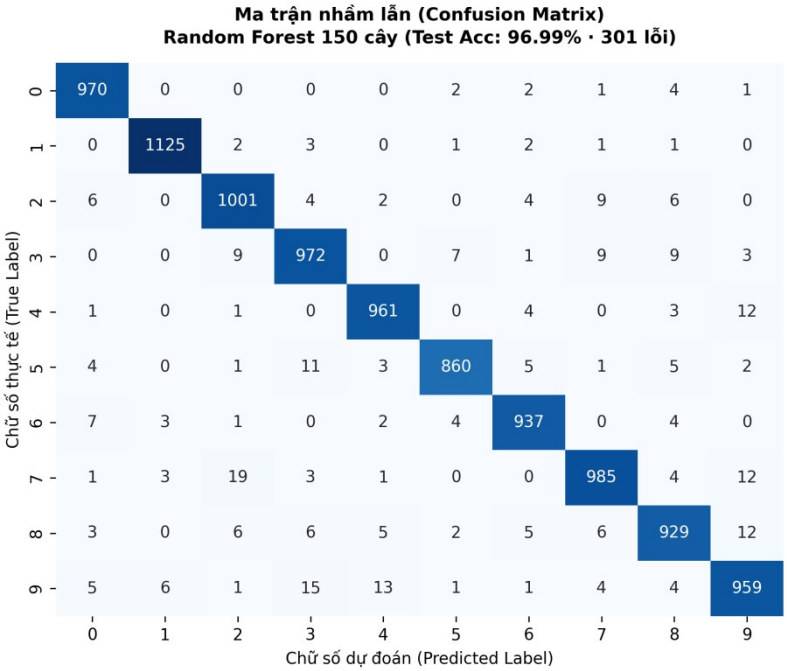
Hình 5.3. Ma trận nhầm lẫn của CNN.



Hình 5.4. Ma trận nhầm lẫn của Bernoulli Naive Bayes trên tập kiểm thử.

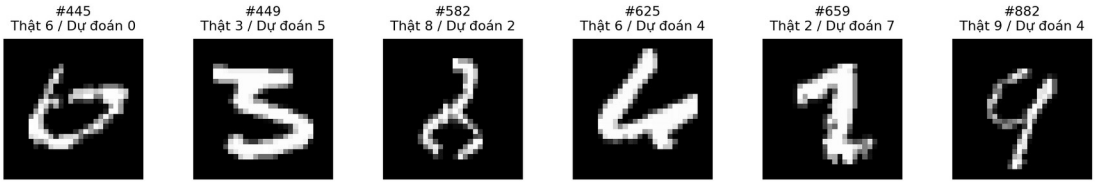


Hình 5.5. Ma trận nhầm lẫn của k-NN trên tập kiểm thử.



Hình 5.6. Ma trận nhầm lẫn của Random Forest trên tập kiểm thử.

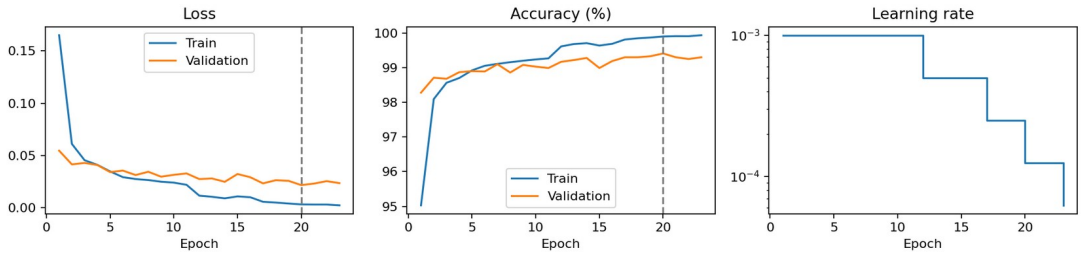
Ở CNN, hàng nhãn thật 9 có 7 ảnh bị nhận thành 4, 3 ảnh thành 5 và 3 ảnh thành 7; hàng nhãn thật 3 có 4 ảnh thành 5. Các số đếm lấy từ ma trận nhầm lẫn của checkpoint epoch 20. Hình 5.7 minh họa sáu mẫu sai lấy trực tiếp từ dự đoán trên tập test, kèm chỉ số mẫu, nhãn thật và nhãn dự đoán. Sự tương đồng về nét cong, nét khép vòng hoặc độ nghiêng là giả thuyết giải thích hình dạng, chưa phải kết luận nguyên nhân cho toàn bộ 56 lỗi.



Hình 5.7. Sáu ảnh CNN nhận dạng sai, kèm chỉ số mẫu và nhãn đối chiếu.

5.4. Đường cong học và lựa chọn checkpoint CNN

Hình 5.8 thể hiện train loss, validation loss, Accuracy và tốc độ học theo từng epoch. Việc validation loss không luôn giảm theo từng epoch là hành vi thường gặp trong tối ưu mini-batch, nên checkpoint tốt nhất được chọn theo cực tiểu của validation loss chứ không dùng kết quả ở epoch cuối. Lần chạy đạt validation loss tốt nhất 0,0218 ở epoch 20, tiếp tục huấn luyện đến epoch 23 rồi dừng sớm; trọng số được phục hồi từ epoch 20 để ghi kết quả 99,41% trên validation và 99,44% trên test.



Hình 5.8. Diễn biến huấn luyện CNN và mốc checkpoint tốt nhất.

Đường học trong một lần chạy cung cấp bằng chứng về cơ chế lựa chọn checkpoint nhưng chưa đủ để kết luận độ ổn định theo seed. Một đánh giá sâu hơn có thể chạy nhiều seed và báo trung bình ± độ lệch chuẩn, hoặc so sánh dự đoán cặp mô hình bằng kiểm định phù hợp. Đây là đề xuất nghiên cứu tiếp theo, **không** phải kết quả đã tiến hành trong phạm vi thực nghiệm hiện tại.

5.5. Trả lời câu hỏi nghiên cứu

RQ1. MNIST có mười lớp gần cân bằng; entropy của train đạt 3,3199 bit. Nền pixel chiếm tỷ lệ rất lớn; hình dạng nét và trọng tâm biến thiên đáng kể. PCA trên 60.000 ảnh gốc giữ 95% phương sai với 154 chiều, trong khi PCA trên 50.000 ảnh đã deskew của nhánh thực nghiệm giữ 113 chiều. t-SNE giúp quan sát cấu trúc lân cận nhưng không trực tiếp là cơ sở chọn mô hình.

RQ2. Trên test chính thức, CNN cao nhất 99,44%; tiếp theo là SVM 98,79%, k-NN 98,06%, Random Forest 96,99%, Naive Bayes 84,46%. Macro F1 tương ứng 99,43%; 98,78%; 98,05%; 96,97%; 84,27%. Ma trận nhầm lẫn CNN cho thấy vẫn tồn tại các lỗi $9 \rightarrow 4$ và $3 \rightarrow 5$ cùng những trường hợp khác, nên không có mô hình nào đạt tuyệt đối 100% trong kết quả được lưu.

RQ3. CNN là lựa chọn hợp lý nếu tiêu chí chính trong lần thực nghiệm này là độ chính xác trên MNIST. Tuy nhiên, đề tài chưa kiểm chứng so sánh độ trễ, dung lượng bộ mô hình, khả năng chống nhiễu ảnh thực tế, chất lượng xác suất hiệu chỉnh hay thống kê nhiều seed. Do đó chưa thể khẳng định CNN là lựa chọn tốt nhất theo mọi ràng buộc triển khai.

CHƯƠNG 6. TRIỂN KHAI MINH HỌA, KHẢ NĂNG TÁI LẠP VÀ HẠN CHẾ

6.1. Trạng thái ứng dụng và phạm vi được xác nhận

Ứng dụng web nhận ảnh chữ số từ bảng vẽ hoặc tệp tải lên và trả kết quả của năm mô hình trong saved_models/. Backend sử dụng cùng checkpoint CNN epoch 20 và cùng bộ PCA 113 chiều với notebook huấn luyện. Khi khởi động, web đọc chỉ số từ results.csv và thông tin huấn luyện từ selected_config.json trong thư mục mô hình. CNN đạt 99,44% với 56 ảnh sai trên tập test MNIST; đây là kết quả của checkpoint đã được đối chiếu bằng dự đoán trên toàn bộ 10.000 ảnh test.

Đầu vào web được chuyển về ảnh xám, xác định vùng chữ số, đổi kích thước và căn tâm trên khung 28 × 28. Pixel chuyển sang float32 trong [0, 1]. CNN nhận tensor 1 × 28 × 28; k-NN/SVM dùng Deskew và PCA đã fit trên 50.000 ảnh train. Đường suy luận SVM sử dụng decision_function khi predict_proba không khả dụng. Điểm từ softmax hoặc khoảng cách lề chưa qua hiệu chuẩn xác suất. Tiền xử lý ảnh người dùng khác việc đưa trực tiếp ảnh MNIST chuẩn vào mô hình, nên kết quả trên web có thể khác kết quả test MNIST.

6.2. Kiểm thử và đối chiếu kết quả

Bộ kiểm thử kiểm tra nạp dữ liệu, phân chia tập, miền giá trị pixel, kích thước đặc trưng, kiến trúc CNN, lịch sử dừng sớm và dự đoán của checkpoint. Các chỉ số trong CSV được tính lại từ predictions.npz để đối chiếu Accuracy, macro Precision, Recall, F1 và số ảnh sai. Kiểm tra API thực hiện với cả năm mô hình, bao gồm trường hợp SVM không cung cấp predict_proba. Kiểm tra HTTP xác nhận endpoint nhận ảnh và trả phân bố điểm trên mười lớp. Các phép thử chức năng này không tạo ra tỷ lệ chính xác mới cho ảnh viết tay ngoài MNIST.

Kết quả kiểm tra xác nhận web nạp cùng bộ mô hình đã đánh giá. Thông tin checkpoint tốt nhất và epoch dừng được đọc từ cấu hình đã lưu, giúp phân biệt checkpoint epoch 20 với thời điểm kết thúc huấn luyện ở epoch 23. Thay đổi trọng số trong một lần huấn luyện mới đồng thời tạo dự đoán, bảng chỉ số và cấu hình mới.

6.3. Giới hạn của thực nghiệm

Thứ nhất, về thiết kế so sánh: chỉ có một cấu hình đại diện cho mỗi thuật toán và một seed; sự khác biệt tiền xử lý giữa các nhánh khiến kết quả phản ánh toàn bộ pipeline. Thực nghiệm hiện tại chưa có nghiên cứu ablation để tách tác động của deskew, PCA, số lượng ảnh train và kiến trúc CNN.

Thứ hai, về thước đo: báo cáo cung cấp Accuracy, macro Precision/Recall/F1 và ma trận nhầm lẫn. Chưa có đo thời gian huấn luyện, độ trễ suy luận, độ lệch chuẩn qua nhiều lần chạy, độ hiệu chỉnh xác suất hoặc tính ổn định dưới biến đổi ảnh. Không đưa ra số liệu cho những tiêu chí chưa thực nghiệm.

Thứ ba, về dữ liệu: MNIST có cấu trúc đã chuẩn hóa. Dữ liệu ghi bằng bút, bút cảm ứng hay chụp qua camera có thể khác màu nền, tỷ lệ, độ nghiêng, nét và nhiễu. Mức chính xác trên test MNIST không được quảng bá thành hiệu năng trên mọi ảnh viết tay trong thực tế.

Thứ tư, về tính tái lập: chạy lại mã CNN trong môi trường CPU đã ghi nhận tái hiện checkpoint epoch 20, dừng sớm ở epoch 23 và kết quả 99,44% trên test. Kết quả này xác nhận lần chạy trong môi trường kiểm tra; độ ổn định qua nhiều seed, hệ điều hành hoặc thiết bị khác chưa được khảo sát.

Thứ năm, các công thức trình bày nguyên lý của mô hình. Chi tiết xử lý khoảng cách k-NN bằng 0 và gamma=scale tuân theo hiện thực scikit-learn. Tài liệu gốc cung cấp cơ sở phương pháp; các số liệu thực nghiệm trong báo cáo lấy từ notebook và bộ mô hình đã lưu.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đề tài đã tổ chức quy trình nhận dạng chữ số viết tay MNIST từ khám phá dữ liệu đến tiền xử lý, huấn luyện và đánh giá năm mô hình. Phân tích EDA cho thấy dữ liệu có mười lớp gần cân bằng, tỷ lệ pixel nền lớn và biến thiên đáng kể về nét viết. Với giao thức đánh giá giữ riêng 10.000 ảnh test, CNN đạt Accuracy cao nhất 99,44%, tương ứng 56 ảnh sai; SVM 98,79%; k-NN 98,06%; Random Forest 96,99% và Naive Bayes 84,46%. Chất lượng của CNN được hỗ trợ bởi ma trận nhầm lẫn và cách chọn checkpoint theo validation loss; đồng thời, một số nhầm lẫn giữa các lớp còn tồn tại.

Thực nghiệm cho phép so sánh các pipeline trên cùng tập test và xác định CNN là mô hình có độ chính xác cao nhất trong cấu hình đã khảo sát. Trọng số, dự đoán, bảng kết quả, báo cáo và ứng dụng web cùng sử dụng checkpoint epoch 20. Hướng phát triển gồm thí nghiệm ablation cho Deskew/PCA, đo độ trễ và kích thước mô hình, chạy nhiều seed, đánh giá ảnh viết tay ngoài MNIST và hiệu chuẩn điểm tin cậy.

TÀI LIỆU THAM KHẢO

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi: 10.1109/5.726791. https://doi.org/10.1109/5.726791

[2] T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. doi: 10.1109/TIT.1967.1053964. https://doi.org/10.1109/TIT.1967.1053964

[3] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995. doi: 10.1007/BF00994018. https://doi.org/10.1007/BF00994018

[4] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001. doi: 10.1023/A:1010933404324. https://doi.org/10.1023/A:1010933404324

[5] Keras, “MNIST digits classification dataset,” *Keras API Documentation*. https://keras.io/api/datasets/mnist/ (truy cập ngày 02/11/2026).

[6] I. T. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, art. 20150202, 2016. doi: 10.1098/rsta.2015.0202. https://doi.org/10.1098/rsta.2015.0202

[7] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008. https://jmlr.org/papers/v9/vandermaaten08a.html

[8] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. doi: 10.1016/j.ipm.2009.03.002. https://doi.org/10.1016/j.ipm.2009.03.002

[9] Scikit-learn developers, “Common pitfalls and recommended practices,” *scikit-learn User Guide*, mục “Data leakage”. https://scikit-learn.org/stable/common_pitfalls.html (truy cập ngày 02/11/2026).

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[11] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016. https://www.deeplearningbook.org/

[12] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*, pp. 8026–8037, 2019. https://papers.neurips.cc/paper/9015-pytorch-an-imperative-stylehigh-performance-deep-learning-library